Coding Standards and Guidelines

In formal reviews, the inspectors are looking for problems and omissions in the code. There are the classic bugs where something just won't work as written. These are best found by careful analysis of the codesenior programmers and testers are great at this.

There are also problems where the code may operate properly but may not be written to meet a specific *standard* or *guideline*. It's equivalent to writing words that can be understood and get a point across but don't meet the grammatical and syntactical rules of the English language. Standards are the established, fixed, have-to-follow-them rulesthe do's and don'ts. Guidelines are the suggested best practices, the recommendations, the preferred way of doing things. Standards have no exceptions, short of a structured waiver process. Guidelines can be a bit loose.

It may sound strange that some piece of software may work, may even be tested and shown to be very stable, but still be incorrect because it doesn't meet some criteria. It's important, though, and there are three reasons for adherence to a standard or guideline:

- **Reliability**. It's been shown that code written to a specific standard or guideline is more reliable and secure than code that isn't.
- **Readability/Maintainability**. Code that follows set standards and guidelines is easier to read, understand, and maintain.
- **Portability**. Code often has to run on different hardware or be compiled with different compilers. If it follows a set standard, it will likely be easieror even completely painlessto move it to a different platform.

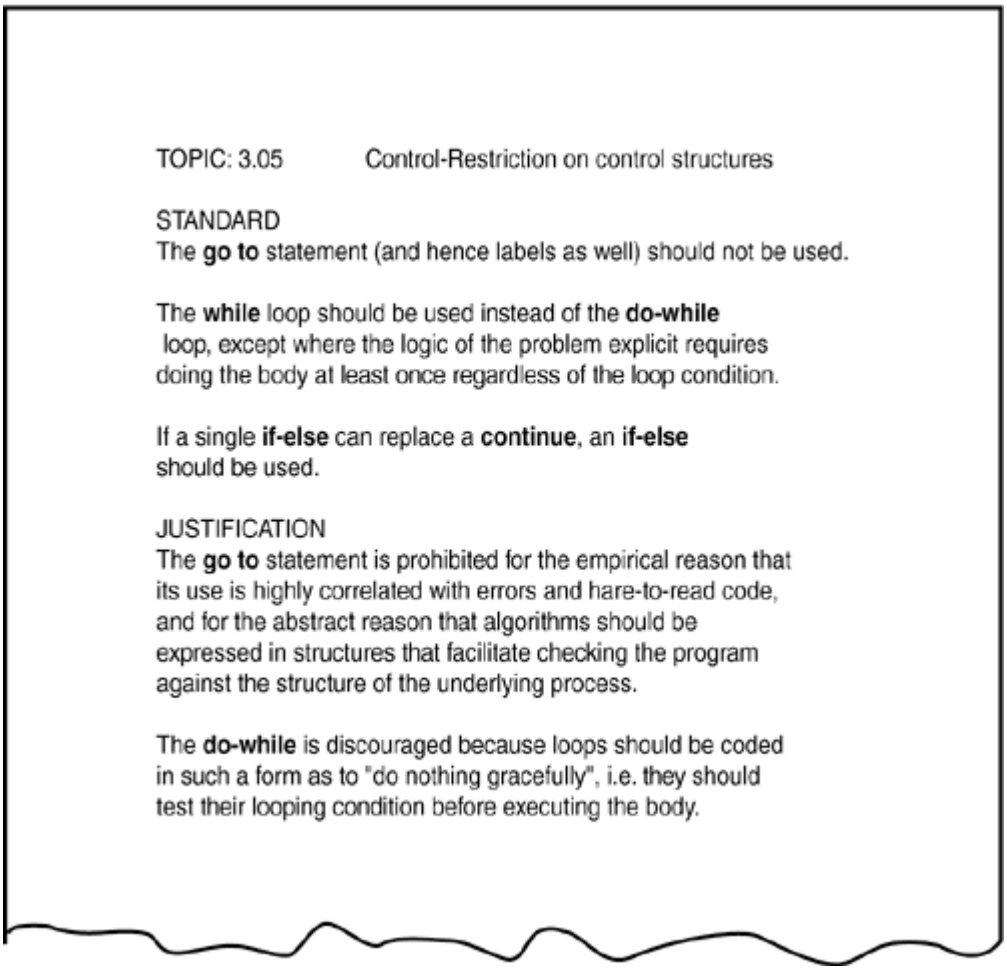
The requirements for your project may range from strict adherence to national or international standards to loose following of internal team guidelines. What's important is that your team has some standards or guidelines for programming and that these are verified in a formal review.

Examples of Programming Standards and Guidelines

Figure 6.2 shows an example of a programming standard that deals with the use of the C language `goto`, `while`, and `if-else` statements. Improper use of these statements often results in buggy code, and most programming standards explicitly set rules for using them.

Figure 6.2. A sample coding standard explains how several language control structures should be used. (Adapted from *C++*

***Programming Guidelines* by Thomas Plum and Dan Saks. Copyright 1991, Plum Hall, Inc.)**



TOPIC: 3.05 Control-Restriction on control structures

STANDARD
The **go to** statement (and hence labels as well) should not be used.

The **while** loop should be used instead of the **do-while** loop, except where the logic of the problem explicit requires doing the body at least once regardless of the loop condition.

If a single **if-else** can replace a **continue**, an **if-else** should be used.

JUSTIFICATION
The **go to** statement is prohibited for the empirical reason that its use is highly correlated with errors and hare-to-read code, and for the abstract reason that algorithms should be expressed in structures that facilitate checking the program against the structure of the underlying process.

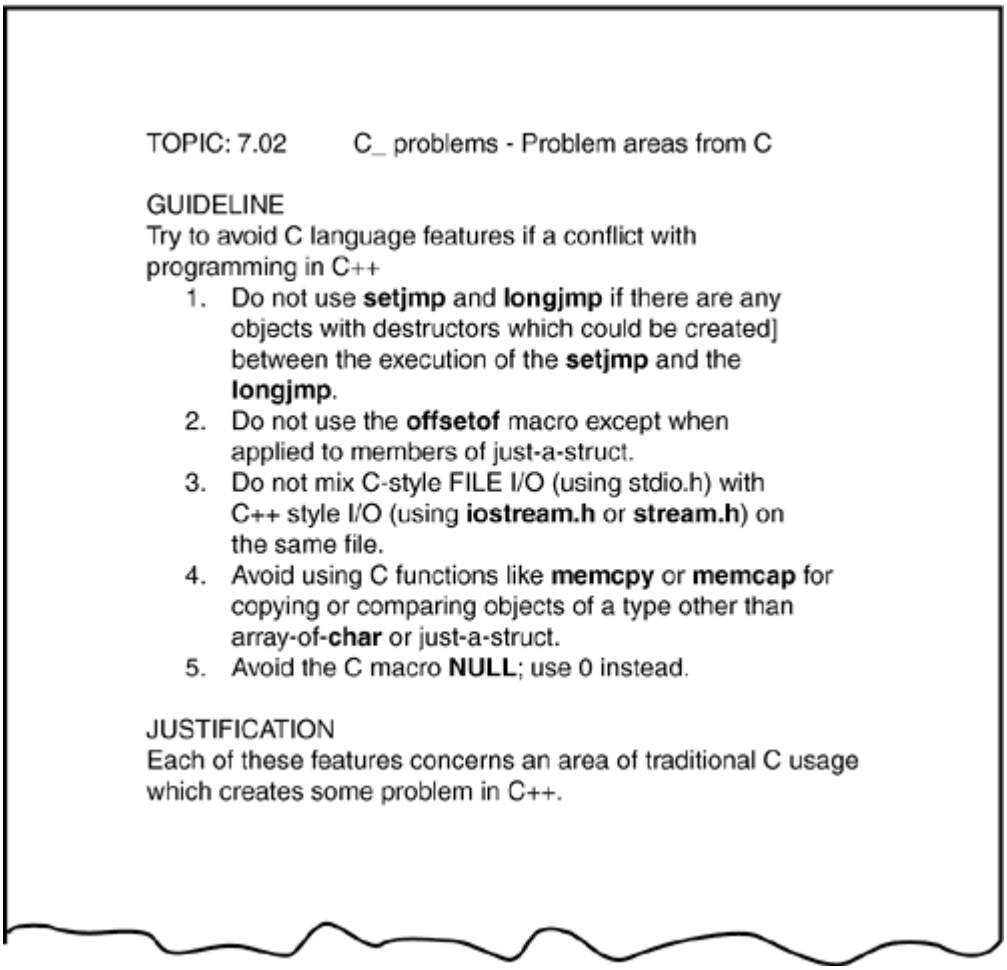
The **do-while** is discouraged because loops should be coded in such a form as to "do nothing gracefully", i.e. they should test their looping condition before executing the body.

The standard has four main parts:

- *Title* describes what topic the standard covers.
- *Standard (or guideline)* describes the standard or guideline explaining exactly what's allowed and not allowed.
- *Justification* gives the reasoning behind the standard so that the programmer understands why it's good programming practice.
- *Example* shows simple programming samples of how to use the standard. This isn't always necessary.

Figure 6.3 is an example of a guideline dealing with C language features used in C++. Note how the language is a bit different. In this case, it starts out with "Try to avoid." Guidelines aren't as strict as standards, so there is some room for flexibility if the situation calls for it.

Figure 6.3. An example of a programming guideline shows how to use certain aspects of C in C++. (Adapted from *C++ Programming Guidelines* by Thomas Plum and Dan Saks. Copyright 1991, Plum Hall, Inc.)



TOPIC: 7.02 C_ problems - Problem areas from C

GUIDELINE
Try to avoid C language features if a conflict with programming in C++

1. Do not use **setjmp** and **longjmp** if there are any objects with destructors which could be created] between the execution of the **setjmp** and the **longjmp**.
2. Do not use the **offsetof** macro except when applied to members of just-a-struct.
3. Do not mix C-style FILE I/O (using stdio.h) with C++ style I/O (using **iostream.h** or **stream.h**) on the same file.
4. Avoid using C functions like **memcpy** or **memcap** for copying or comparing objects of a type other than array-of-**char** or just-a-struct.
5. Avoid the C macro **NULL**; use 0 instead.

JUSTIFICATION
Each of these features concerns an area of traditional C usage which creates some problem in C++.

IT'S A MATTER OF STYLE

There are standards, there are guidelines, and then there is style. From a software quality and testing perspective, style doesn't matter.

Every programmer, just like every book author and artist, has his or her own unique style. The rules may be followed, the language usage may be consistent, but it's still easy to tell who created what software.

That differentiating factor is style. In programming, it could be how verbose the commenting is or how the variables are named. It could be what indentation scheme is used in the loop constructs. It's the look and feel of the code.

Some teams do institute standards and guidelines for style aspects (such as indenting) so that the look and feel of the code doesn't become too random. As a software tester, when performing formal reviews on a piece of software, test and comment only on things that are wrong, missing, or don't adhere to the team's accepted standards or guidelines. Ask yourself if what you're about to report is really a problem or just difference of opinion, a difference of style. The latter isn't a bug.

Obtaining Standards

If your project, because of its nature, must follow a set of programming standards, or if you're just interested in examining your software's code to see how well it meets a published standard or guideline, several sources are available for you to reference.

National and international standards for most computer languages and information technology can be obtained from:

- American National Standards Institute (ANSI), www.ansi.org
- International Engineering Consortium (IEC), www.iec.org
- International Organization for Standardization (ISO), www.iso.ch
- National Committee for Information Technology Standards (NCITS), www.ncits.org

There are also documents that demonstrate programming guidelines and best practices available from professional organizations such as

- Association for Computing Machinery (ACM), www.acm.org
- Institute of Electrical and Electronics Engineers, Inc (IEEE), www.ieee.org

You may also obtain information from the software vendor where you purchased your programming software. They often have published standards and guidelines available for free or for a small fee.

Generic Code Review Checklist

The rest of this chapter on static white-box testing covers some problems you should look for when verifying software for a formal code review. These checklists[1] are in addition to comparing the code against a standard or a guideline and to making sure that the code meets the project's design requirements.

[1] *These checklist items were adapted from* Software Testing in the Real World: Improving the Process, *pp. 198-201. Copyright 1995 by Edward Kit. Used by permission of Pearson Education Limited, London. All rights reserved.*

To really understand and apply these checks, you should have some programming experience. If you haven't done much programming, you might find it useful to read an introductory book such as *Sams Teach Yourself Beginning Programming in 24 Hours* by Sams Publishing before you attempt to review program code in detail.

Data Reference Errors

Data reference errors are bugs caused by using a variable, constant, array, string, or record that hasn't been properly declared or initialized for how it's being used and referenced.

- Is an uninitialized variable referenced? Looking for omissions is just as important as looking for errors.
- Are array and string subscripts integer values and are they always within the bounds of the array's or string's dimension?
- Are there any potential "off by one" errors in indexing operations or subscript references to arrays? Remember the code in Listing 5.1 from Chapter 5.
- Is a variable used where a constant would actually work betterfor example, when checking the boundary of an array?
- Is a variable ever assigned a value that's of a different type than the variable? For example, does the code accidentally assign a floating-point number to an integer variable?
- Is memory allocated for referenced pointers?
- If a data structure is referenced in multiple functions or subroutines, is the structure defined identically in each one?

NOTE

Data reference errors are the primary cause of buffer overrunsthe bug behind many software security issues. Chapter 13, "Testing for Software Security," covers this topic in more detail.

Data Declaration Errors

Data declaration bugs are caused by improperly declaring or using variables or constants.

- Are all the variables assigned the correct length, type, and storage class? For example, should a variable be declared as a string instead of an array of characters?
- If a variable is initialized at the same time as it's declared, is it properly initialized and consistent with its type?
- Are there any variables with similar names? This isn't necessarily a bug, but it could be a sign that the names have been confused with those from somewhere else in the program.
- Are any variables declared that are never referenced or are referenced only once?
- Are all the variables explicitly declared within their specific module? If not, is it understood that the variable is shared with the next higher module?

Computation Errors

Computational or calculation errors are essentially bad math. The calculations don't result in the expected result.

- Do any calculations that use variables have different data types, such as adding an integer to a floating-point number?
- Do any calculations that use variables have the same data type but are different lengthsadding a byte to a word, for example?
- Are the compiler's conversion rules for variables of inconsistent type or length understood and taken into account in any calculations?

- Is the target variable of an assignment smaller than the right-hand expression?
- Is overflow or underflow in the middle of a numeric calculation possible?
- Is it ever possible for a divisor/modulus to be zero?
- For cases of integer arithmetic, does the code handle that some calculations, particularly division, will result in loss of precision?
- Can a variable's value go outside its meaningful range? For example, could the result of a probability be less than 0% or greater than 100%?
- For expressions containing multiple operators, is there any confusion about the order of evaluation and is operator precedence correct? Are parentheses needed for clarification?

Comparison Errors

Less than, greater than, equal, not equal, true, false. Comparison and decision errors are very susceptible to boundary condition problems.

- Are the comparisons correct? It may sound pretty simple, but there's always confusion over whether a comparison should be less than or less than or equal to.
- Are there comparisons between fractional or floating-point values? If so, will any precision problems affect their comparison? Is 1.00000001 close enough to 1.00000002 to be equal?
- Does each Boolean expression state what it should state? Does the Boolean calculation work as expected? Is there any doubt about the order of evaluation?
- Are the operands of a Boolean operator Boolean? For example, is an integer variable containing integer values being used in a Boolean calculation?

Control Flow Errors

Control flow errors are the result of loops and other control constructs in the language not behaving as expected. They are usually caused, directly or indirectly, by computational or comparison errors.

- If the language contains statement groups such as `begin...end` and `do...while`, are the `end`s explicit and do they match their appropriate groups?
- Will the program, module, subroutine, or loop eventually terminate? If it won't, is that acceptable?
- Is there a possibility of premature loop exit?
- Is it possible that a loop never executes? Is it acceptable if it doesn't?
- If the program contains a multiway branch such as a `switch...case` statement, can the index variable ever exceed the number of branch possibilities? If it does, is this case handled properly?
- Are there any "off by one" errors that would cause unexpected flow through the loop?

Subroutine Parameter Errors

Subroutine parameter errors are due to incorrect passing of data to and from software subroutines.

- Do the types and sizes of parameters received by a subroutine match those sent by the calling code? Is the order correct?
- If a subroutine has multiple entry points (yuck), is a parameter ever referenced that isn't associated with the current point of entry?
- If constants are ever passed as arguments, are they accidentally changed in the subroutine?
- Does a subroutine alter a parameter that's intended only as an input value?
- Do the units of each parameter match the units of each corresponding argumentEnglish versus metric, for example?
- If global variables are present, do they have similar definitions and attributes in all referencing subroutines?

Input/Output Errors

These errors include anything related to reading from a file, accepting input from a keyboard or mouse, and writing to an output device such as a printer or screen. The items presented here are very simplified and generic. You should adapt and add to them to properly cover the software you're testing.

- Does the software strictly adhere to the specified format of the data being read or written by the external device?
- If the file or peripheral isn't present or ready, is that error condition handled?
- Does the software handle the situation of the external device being disconnected, not available, or full during a read or write?
- Are all conceivable errors handled by the software in an expected way?
- Have all error messages been checked for correctness, appropriateness, grammar, and spelling?

Other Checks

This best-of-the-rest list defines a few items that didn't fit well in the other categories. It's not by any means complete, but should give you ideas for specific items that should be added to a list tailored for your software project.

- Will the software work with languages other than English? Does it handle extended ASCII characters? Does it need to use Unicode instead of ASCII?
- If the software is intended to be portable to other compilers and CPUs, have allowances been made for this? Portability, if required, can be a huge issue if not planned and tested for.
- Has compatibility been considered so that the software will operate with different amounts of available memory, different internal hardware such as graphics and sound cards, and different peripherals such as printers and modems?
- Does compilation of the program produce any "warning" or "informational" messages? They usually indicate that something questionable is being done. Purists would argue that any warning message is unacceptable.

Summary

Examining the codestatic white-box testinghas proven to be an effective means for finding bugs early. It's a task that requires a great deal of preparation to make it a productive exercise, but many studies have shown that the time spent is well worth the benefits gained. To make it even more attractive, commercial software products, known as *static analyzers*, are available to automate a great deal of the work. The software reads in a program's source files and checks them against published standards and your own customizable guidelines. Compilers have also improved to the point that if you enable all their levels of error checking, they will catch many of the problems listed previously in the generic code review checklist. Some will even disallow use of functions with known security issues. These tools don't eliminate the tasks of code reviews or inspectionsthey just make it easier to accomplish and give testers more time to look even deeper for bugs.

If your team currently isn't doing testing at this level and you have some experience at programming, you might try suggesting it as a process to investigate. Programmers and managers may be apprehensive at first, not knowing if the benefits are that greatit's hard to claim, for example, that finding a bug during an inspection saved your project five days over finding it months later during black-box testing. But, static white-box testing is gaining momentum, and in some circles, projects can't ship reliable software without it.

Quiz

These quiz questions are provided for your further understanding. See Appendix A, "Answers to Quiz Questions," for the answersbut don't peek!

1: Name several advantages to performing static white-box testing.

2: **True or False:** Static white-box testing can find missing items as well as problems.

3: What key element makes formal reviews work?

4: Besides being more formal, what's the big difference between inspections and other types of reviews?

5: If a programmer was told that he could name his variables with only eight characters and the first character had to be capitalized, would that be a standard or a guideline?

6: Should you adopt the code review checklist from this chapter as your team's standard to verify its code?

7: A common security issue known as a buffer overrun is in a class of errors known as what? They are caused by what?